

Theory of Computation

303105306

Dr. Sandeep Wadekar

Assistant Professor

Department of Artificial Intelligence & Data Science

UNIT-4

Turing machines

Content

Sr. No.	Topic	Page No.
1	Turing Machine	3
2	Definition of Turing Machine	7
3	Examples of Turing Machine	8
4	Turing Recognizable & Decidable Language	14
5	Closure Properties	15
6	Variants of Turing Machine	18
7	Multi-track Turing Machine	19
8	Multi-tape Turing Machine	20
9	Multi-head Turing Machine	21
10	Deterministic vs Non-deterministic TM	22
11	Unrestricted Grammar	23
12	Type-0 Grammar and TM Equivalence	23
13	Turing Machine Enumerator	25
14	References	27

Understanding of Machines

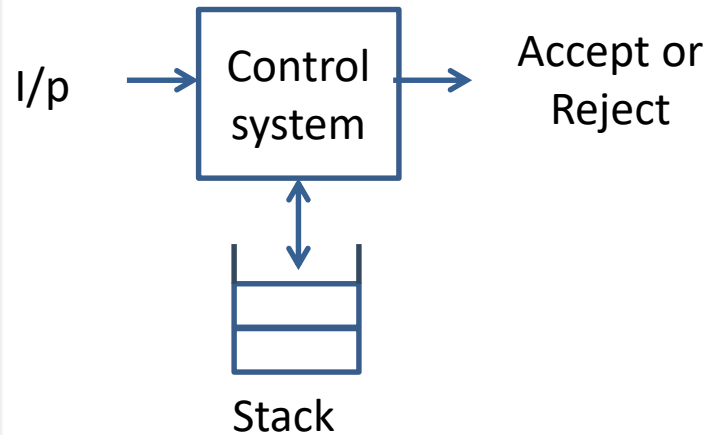
Finite Automata

- Input Strings

a b b a
→

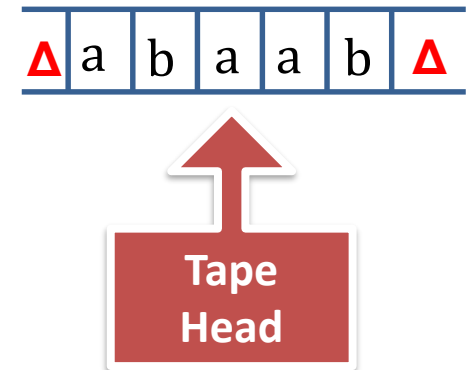
Pushdown Automata

- Input Strings & Stack

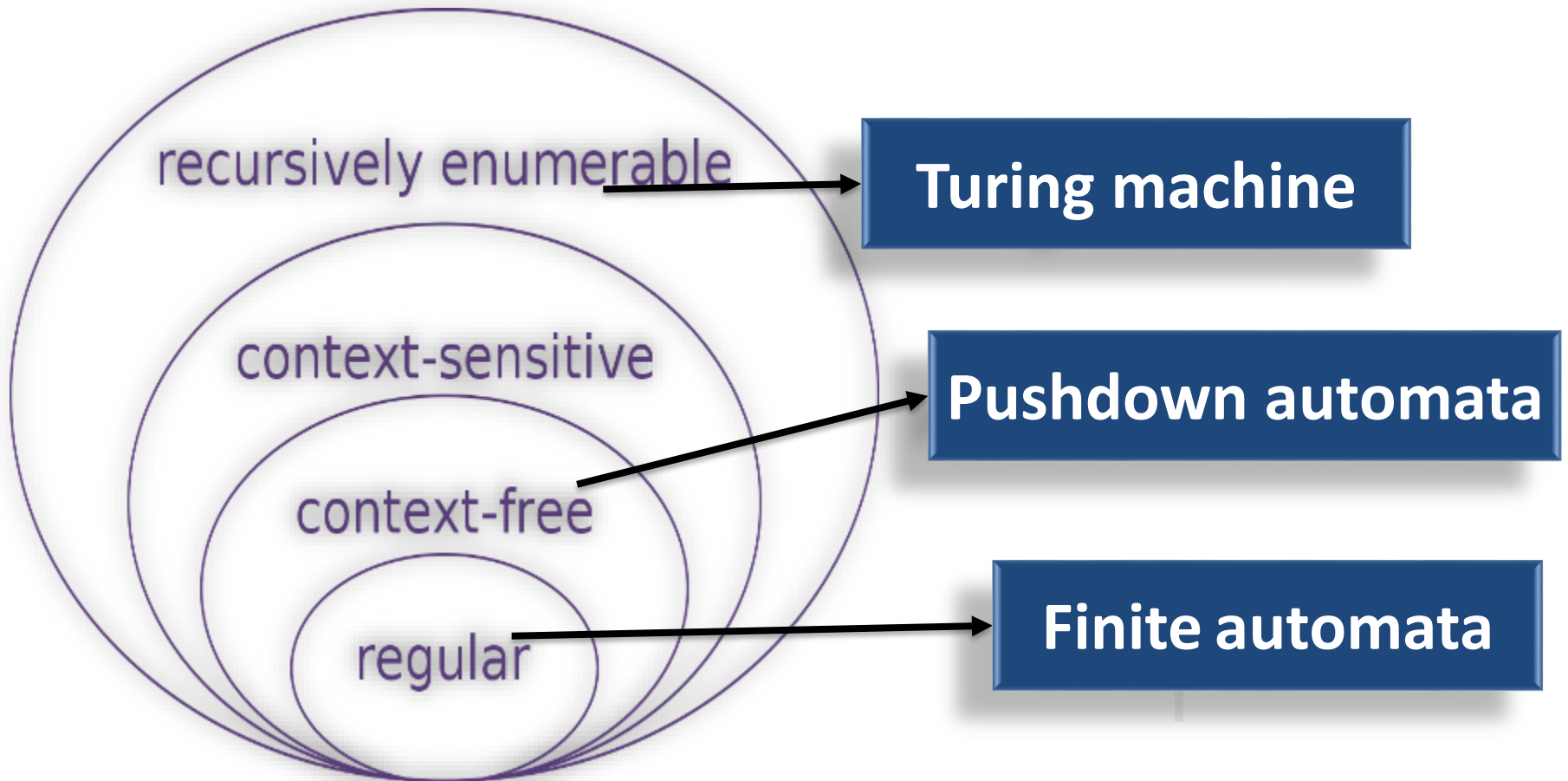


Turing Machine

- Input Strings
- Magnetic Tape
- Special symbol
- Control system

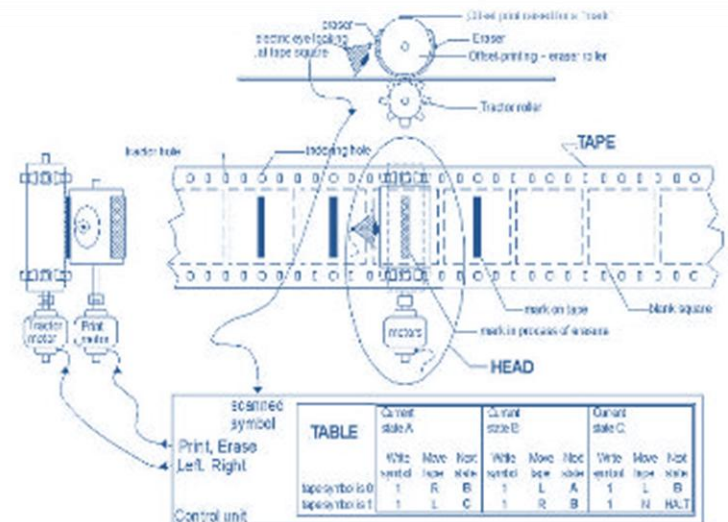


Chomsky-hierarchy



Turing Machine

- ❑ A Turing Machine (TM) has finite-state control (like PDA), and an infinite read-write tape.
- ❑ The tape serves as both input and unbounded storage device.
- ❑ The tape is divided into cells, and each cell holds one symbol from the tape alphabet like 1 & 0.
- ❑ There is a special blank symbol B. At any instant, all but finitely many cells hold B.
- ❑ Tape head sees only one cell at any instant. The contents of this cell and the current state determine the next move of the TM.



A fanciful mechanical Turing machine's TAPE and HEAD. The TABLE instructions might be on another "read only" tape, or perhaps on punch-cards. Usually a "finite state machine" is the model for the TABLE.

Contd.

- At any point of time, the head which is positioned over one of the squares on the tape. With this head, the machine can execute three very simple operations:
 - **Read** the symbol under the head on the square.
 - **Edit the symbol** by updating symbol or erasing it.
 - **Shift the head to left or right** by only one square, So that the machine can read and edit or erase the symbol of a neighboring square.

Definition of Turing machine

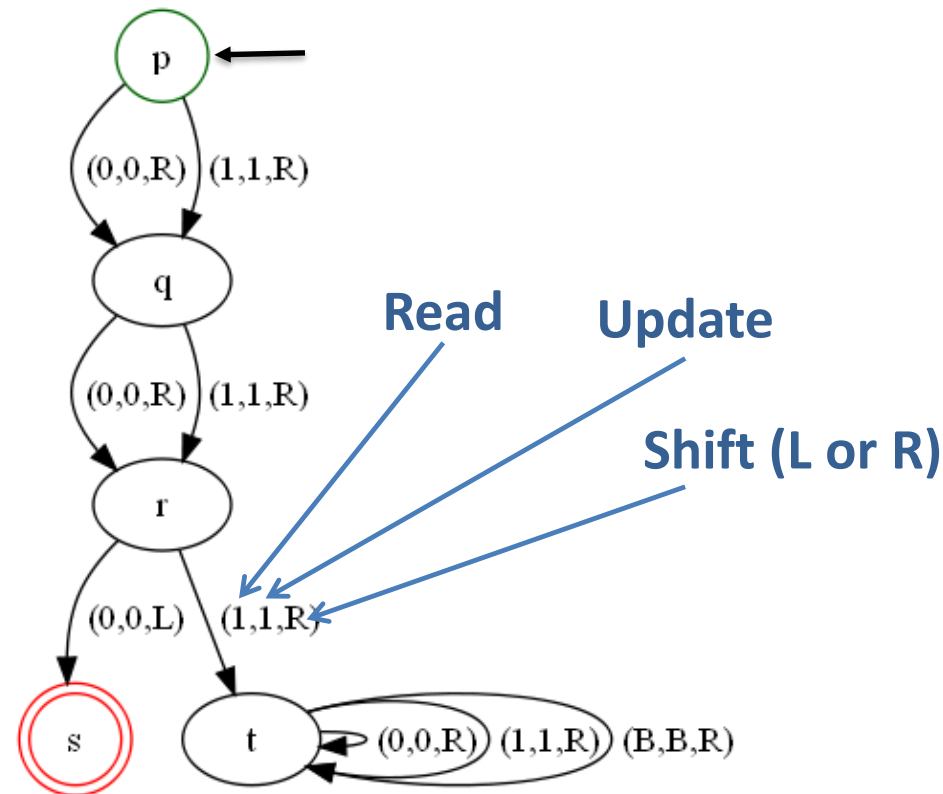
- Formally, a Turing machines a 7- tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where
 - Q is the finite set of states of the finite control
 - Σ is the finite set of input symbols.
 - Γ is the finite set of tape symbols; $\Sigma \subset \Gamma$.
 - $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function, which is a partial function.
 - $q \in Q$ is the start state.
 - $B \in \Gamma$ is the blank symbol; $B \notin \Sigma$.
 - $F \subseteq Q$ is the set of final or accepting states.

Examples

• TM that checks its third symbol is 0, accepts if so, and runs forever, if not (note it never rejects).

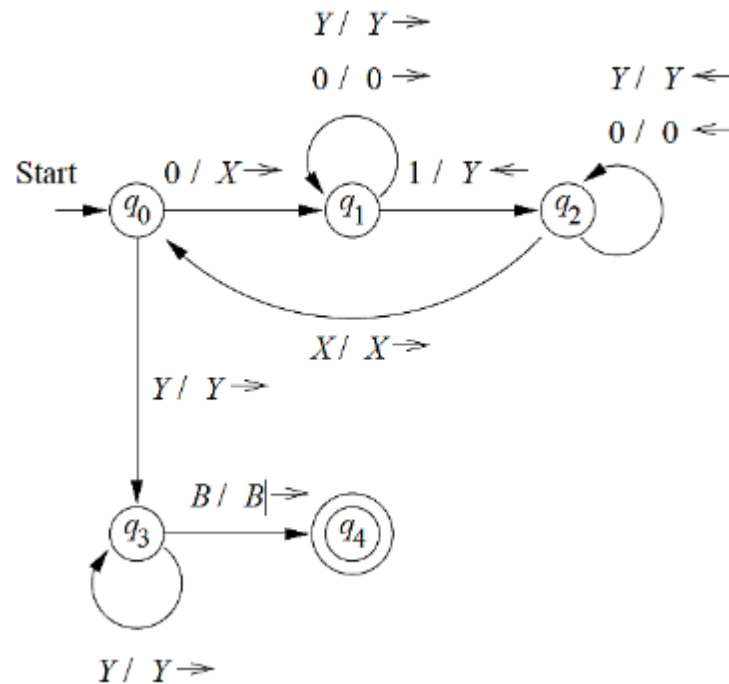
– $M = (\{p, q, r, s, t\}, \{0, 1\}, \{0, 1, B\}, p, s, d)$

- $\delta(p, X) = (q, X, R)$ for $X=0, 1$
- $\delta(q, X) = (r, X, R)$ for $X=0, 1$
- $\delta(r, 0) = (s, 0, L)$
- $\delta(r, 1) = (t, 1, R)$
- $\delta(t, X) = (t, X, R)$ for $X=0, 1, B$



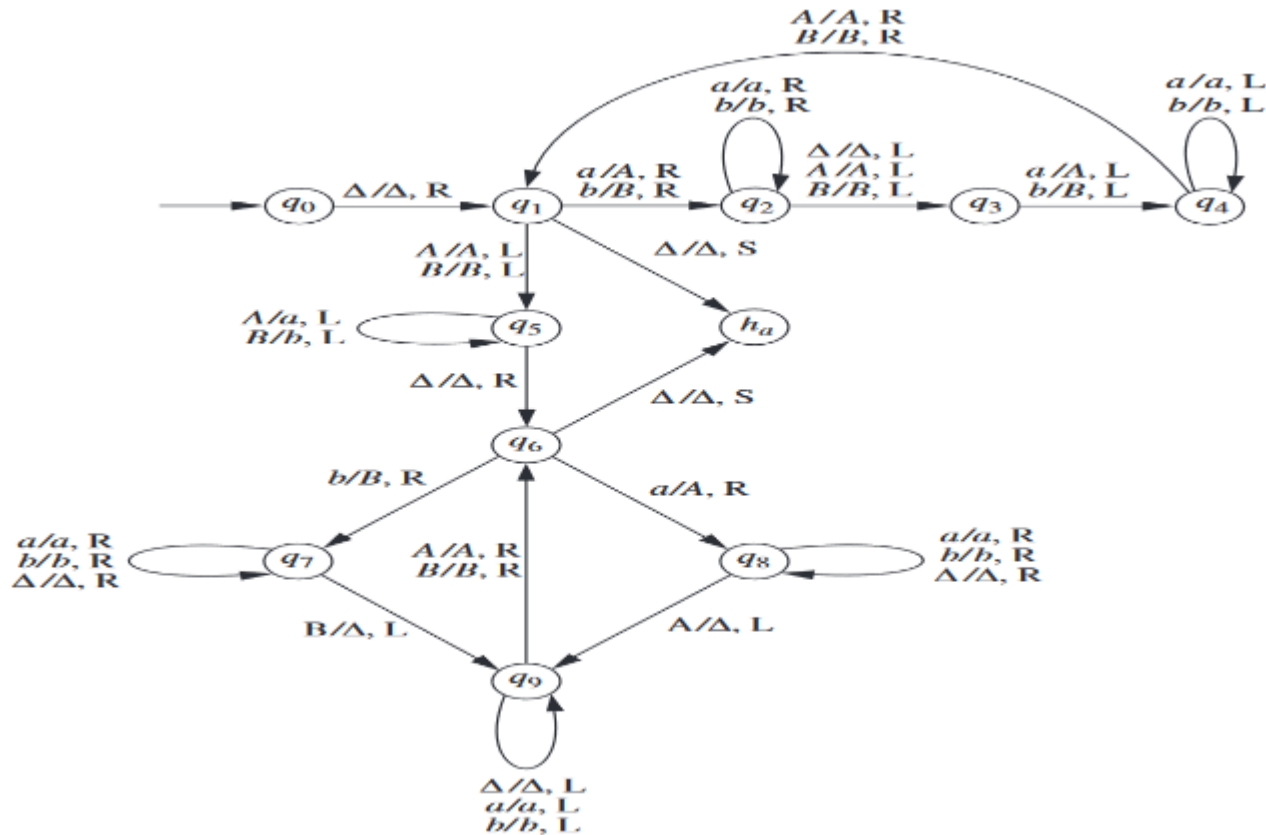
Examples

- TM for $\{0^n 1^n : n \geq 1\}$
 - $M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \pm, q_0, B, \{q_4\})$



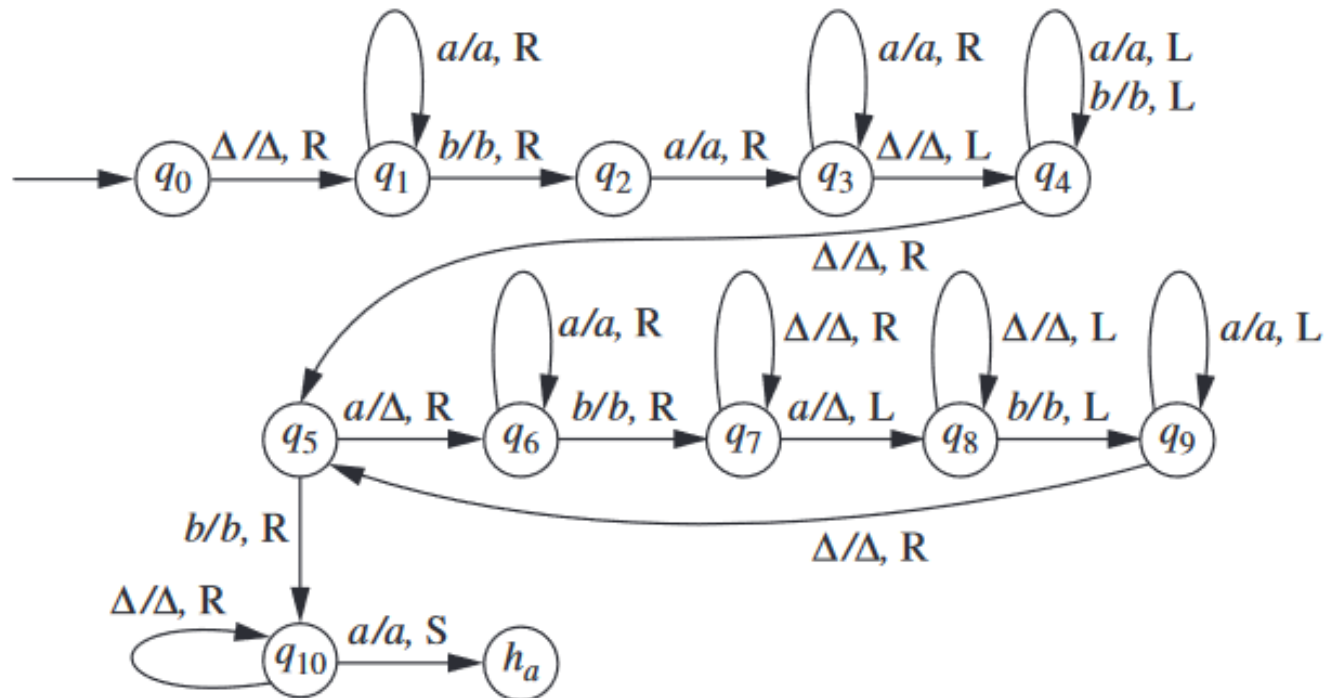
Examples

- A Turing machine accepting $\{xx \mid x \in \{a, b\}^*\}$



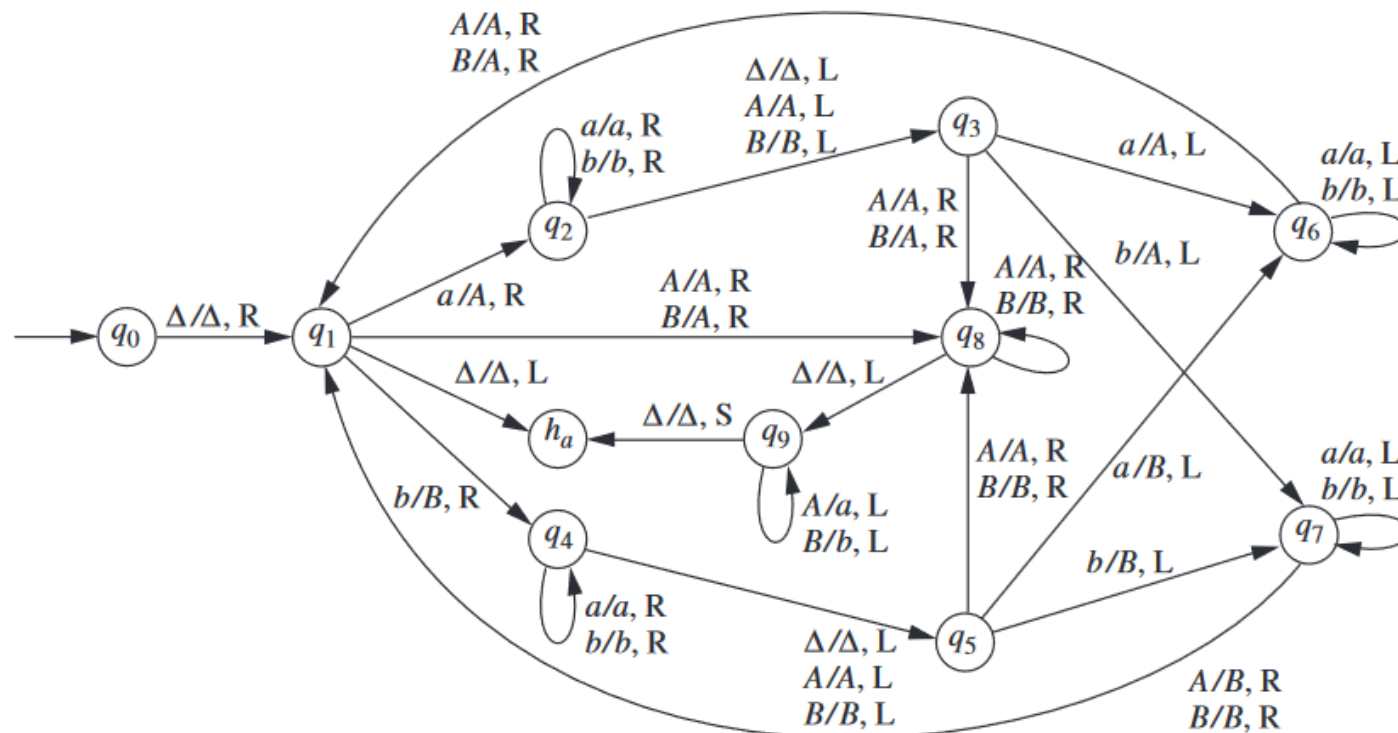
Examples

- A Turing machine accepting $\{aibaj \mid 0 \leq i < j\}$



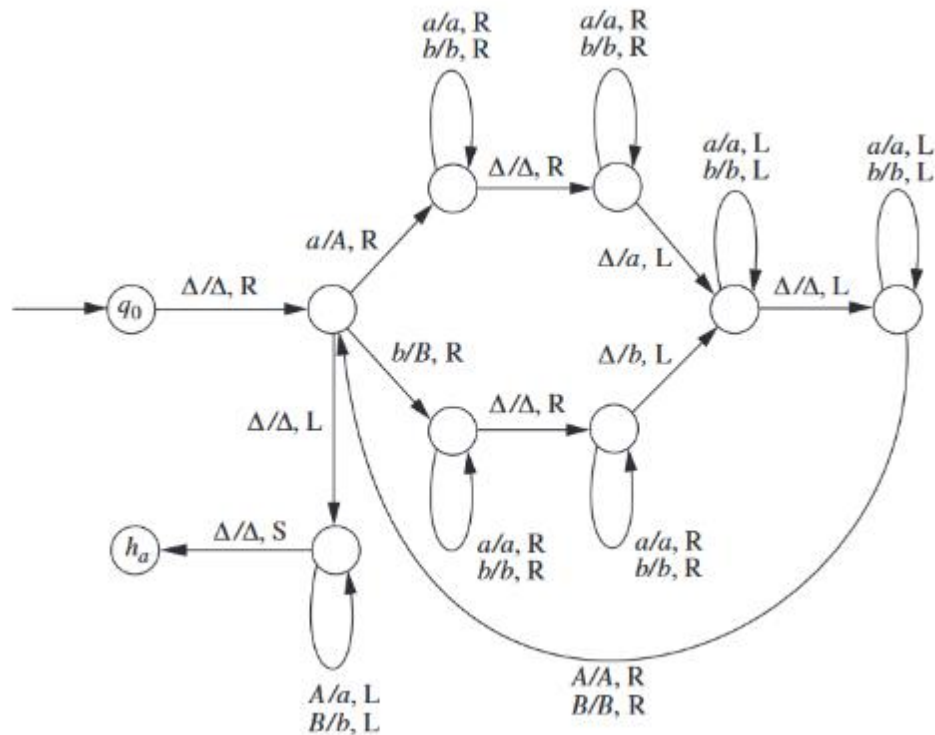
Examples

- A Turing machine computing the reverse function.



Examples

- A Turing machine to copy strings.



Turing Recognizable and Decidable

- A Turing machine M recognizes language L if $L=L(M)$. We say L is Turing-recognizable (or simply recognizable) if there is a TM M such that $L=L(M)$.
- A Turing machine decides language L if $L=L(M)$ and M halts on all inputs. We say L is decidable if there is a TM M that decides L .
- Both types of machine halt in the Accept state on strings that are in the language.
 - A Decider also halts if the string is not in the language.
 - A Recognizer MAY or MAY NOT halt on strings that are not in the language.
- On all input:
 - A Decider MUST halt (in Accept or Reject state)
 - A Recognizer MAY or MAY NOT halt on some strings

Closure properties

-Union

Both decidable and Turing recognizable languages are closed under union.

- For decidable languages the proof is easy. Suppose L_1 and L_2 are two decidable languages accepted by halting TMs M_1 and M_2 respectively. The machine for $L_1 \cup L_2$ is designed as follows:
 - Given an input x , simulate M_1 on x . If M_1 accepts then accept, else simulate M_2 on x . If M_2 accepts then accept else reject.
- Now suppose L_1 and L_2 are two Turing recognizable languages accepted by TMs M_1 and M_2 respectively. Since L_1 and L_2 are Turing recognizable languages, there fore for strings that do not belong to these languages, the corresponding machines may not even halt. The previous strategy will not work because we can have a scenario where M_2 accepts x but M_1 loops forever.

Contd.

- Concatenation

- Both decidable and Turing recognizable languages are closed under concatenation.
- I will give the proof for Turing recognizable languages. The proof for decidable languages is similar. Let L_1 and L_2 be two Turing recognizable languages. Given an input w , use non determinism and guess a partition w (say $w=xy$). Now run the respective Turing machines of L_1 and L_2 on x and y respectively. If both accepts then accept else reject.

- Star

- Both decidable and Turing recognizable languages are closed under star operation.
- This is also similar to concatenation. Non deterministically first guess a number k , and then guess a k partition of the given input. Now for each string in the partition, check whether it belongs to the original language.

Contd.

- Intersection

- Both decidable and Turing recognizable languages are closed under intersection. Run the TMs of both the languages on the given input. Accept if and only if both the machines accept. In the case of intersection we can run the TMs of L_1 and L_2 one after the other (as opposed to union).

- Complementation

- Decidable languages are closed under complementation. To design a machine for the complement of a language L , we can simulate the machine for L on an input. If it accepts then accept and vice versa.

Variants of Turing machines

1. Multiple track Turing Machine
2. Two-way infinite Tape Turing Machine
3. Multi-tape Turing Machine
4. Multi-tape Multi-head Turing Machine
5. Multi-dimensional Tape Turing Machine
6. Multi-head Turing Machine
7. Non-deterministic Turing Machine

Contd.

1. **Multiple track Turing Machine:**

- A k-track Turing machine (for some $k > 0$) has k-tracks and one R/W head that reads and writes all of them one by one.
- A k-track Turing Machine can be simulated by a single track Turing machine

2. **Two-way infinite Tape Turing Machine:**

- Infinite tape of two-way infinite tape Turing machine is unbounded in both directions left and right.
- Two-way infinite tape Turing machine can be simulated by one-way infinite Turing machine (standard Turing machine).

Contd.

3. **Multi-tape Turing Machine:**

- It has multiple tapes and controlled by a single head.
- The Multi-tape Turing machine is different from k-track Turing machine but expressive power is same.
- Multi-tape Turing machine can be simulated by single-tape Turing machine.

4. **Multi-tape Multi-head Turing Machine:**

- The multi-tape Turing machine has multiple tapes and multiple heads
- Each tape controlled by separate head. Multi-Tape Multi-head Turing machine can be simulated by standard Turing machine.

5. **Multi-dimensional Tape Turing Machine:**

- It has multi-dimensional tape where head can move any direction that is left, right, up or down. Multi dimensional tape Turing machine can be simulated by one-dimensional Turing machine

Contd.

6. **Multi-head Turing Machine:**

- A multi-head Turing machine contain two or more heads to read the symbols on the same tape.
- In one step all the heads sense the scanned symbols and move or write independently. Multi-head Turing machine can be simulated by single head Turing machine.

7. **Non-deterministic Turing Machine:**

- A non-deterministic Turing machine has a single, one way infinite tape. For a given state and input symbol has atleast one choice to move (finite number of choices for the next move), each choice several choices of path that it might follow for a given input string.
- A non-deterministic Turing machine is equivalent to deterministic Turing machine.

Deterministic vs. non-determinism Turing Machines

- Deterministic TM's owe their name to the fact that each computation can be viewed as a linear ordered sequence (finite or infinite) of configurations. The first element contains the initial state and each configuration C' follows directly configuration C in the sequence if the TM can change C into C' as a consequence of a legal move of the machine.
- Nondeterministic machines are not restricted to such ordered computations. At each moment in time a non-deterministic TM may be allowed to make several moves (or equivalently to choose among a number of possibilities). Hence its computation may “branch” in many possible ways. The resulting computation is best described therefore as a rooted tree of configurations. Each deterministic computation corresponding to a particular sequence of choices is called a computation path.

Unrestricted grammars.

- Definition: an unrestricted grammar is a 4-tuple (T, N, P, S) , consisting of:
 - T = set of terminals (the legal "tokens" of the language)
 - N = set of non-terminals (aka variables)
 - P = as set of productions, each of the form:
$$v \rightarrow w$$

where v and w are strings consisting of non-terminals and terminals.
 - S = a special nonterminal called the start symbol.

This type of grammar is also known as a **Type 0** grammar. It is equivalent to a **Turing Machine**. Given a Type 0 grammar, there exists a TM that can "recognize" the same language as that grammar. And, conversely, a Type 0 grammar can always be found for the language recognized or generated by any TM.

Example

- Example: Grammar for $L = \{ ww : w \in (a, b)^* \}$
- $S \rightarrow S'Z$
- $S' \rightarrow aS'A$
- $S' \rightarrow bS'B$
- $S' \rightarrow \emptyset$

TMs Enumerator.

An enumerator is multi-tape Turing Machine, with a special output tape which is write-only.

- Write only means (a) symbol on output tape does not affect transitions, and (b) tape head only moves right.
- Initially all tapes blank (no input). During computation the machine adds symbols to the output tape. Output considered to be a list of words (separated by special symbol #).
- **Definition:** An enumerator M is said to enumerate a string W if and only if at some point M writes a word W on the output tape.

Contd.

- Enumerate all the strings for $L=\{a^n b^n c^n \mid n \geq 0\}$

Solution. The L can be generated by two-tape TM E with following steps:

- (1) Write # # on tape 1 and 2 for ϵ .
- (2) Add a on tape 2,
- (3) Copy a equal to size of tape 2 on tape 1, then by same size the b is written on tape 1, then by same size c is written tape 1, followed with #
- (4) go to step 2.

References

1. **Introduction to Automata theory, languages and Computation** By John E. Hopcroft, Rajiv Motwani and Jeffery D. Ullman | Pearson
2. **Elements of the Theory of Computation** Harry R. Lewis and Christos H. Papadimitriou | Pearson Education Asia
3. **Introduction to the Theory of Computation** Michael Sipser | PWS Publishing
4. **Introduction to Languages and the Theory of Computation** John C. Martin | McGraw Hill
5. **Automata and Computability** Dexter C. Kozen | Springer

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

